

Storage and Sessions

Storefront Next uses a **server-only** authentication architecture. All Shopper Login and API Access Service (SLAS) token management—guest login, refresh, registered login, social and passwordless flows, and cookie writing—happens in a single server middleware, `auth.server.ts`, that runs on every request. Tokens never reach the browser. Client components receive only a small, non-sensitive slice of session data through the `useAuth()` hook.

Contents

Architecture Overview

[Cookie Architecture](#)[Configuration](#)[Accessing Session Data](#)[Updating and Destroying Auth](#)[Authentication Flows](#)[Hydration](#)[Type Safety](#)[Best Practices](#)

Architecture Overview

Storefront Next auth has two layers and a single source of truth:

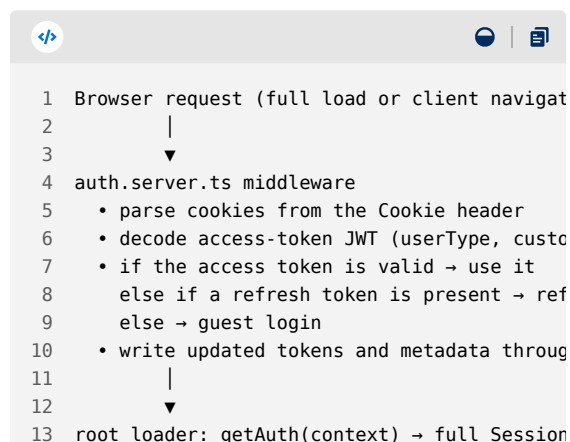
1. Server middleware

(`middlewares/auth.server.ts`) runs on every request—both full page loads and React Router client navigations, which re-invoke the server loaders. It reads auth cookies from the `Cookie` header, validates and refreshes tokens, performs guest login when needed, and writes updated cookies through `Set-Cookie` headers.

2. React context (`providers/auth.tsx`) exposes a `PublicSessionData` object (no tokens) to components through the `useAuth()` hook. The root loader extracts this slice with `getPublicSessionData(session)` and passes it to the provider.

Route modules use only the server `loader` and `action`—never `clientLoader` or `clientAction`.

Request Flow





Cookie Architecture

Storefront Next stores all auth state across separate cookies, each with its own purpose, expiry, and value source. **Every auth cookie is `HttpOnly`**. The browser sends them on each request—so the server middleware and hybrid B2C Commerce can both read them—but client-side JavaScript can't access them. The client gets its auth state from the serialized loader data, not by reading cookies. This architecture protects every token from cross-site scripting (XSS) attacks.

Cookie	Purpose	User type
<code>cc-nx-g</code>	Guest refresh token	Guest
<code>cc-nx</code>	Registered refresh token	Registered
<code>cc-at</code>	Access token	Both
<code>usid</code>	User session ID (mirrors JWT <code>sub</code> → <code>usid</code>)	Both
<code>enc_user_id</code>	Encoded user ID	Registered
<code>idp_access_token</code>	IDP access token (social login)	Both
<code>id_token</code>	OIDC ID token	Both
<code>idp_refresh_token</code>	IDP refresh token (social login)	Both
<code>dw_dnt</code>	Tracking consent preference (value = <code>TrackingConsent</code> enum)	Both
<code>dwsid</code>	Hybrid storefront session ID (B2C)	Both



cc-cv	OAuth2 PKCE code verifier	Both
cc-auth-recover	401-recovery loop guard	Both

Note

Value source. Token strings come verbatim from the SLAS `TokenResponse` body. The session facts—`userId`, `customerId`, `usid`, `accessTokenExpiry`, and tracking consent—are decoded from the access-token JWT rather than read from the body, so they can never drift from the token. The `dwsid` cookie is the exception: it comes from the SLAS response's `Set-Cookie` header.

`customerId` is not a cookie. It's derived per request from the access-token JWT and exposed through `getAuth()` on the server and `useAuth()` on the client. The logout and error paths also clear any legacy `customer_id` cookie left by older versions.

Key Design Decisions

- **Mutually exclusive refresh tokens:** Only one of `cc-nx-g` (guest) or `cc-nx` (registered) exists at a time. On a user-type transition, the middleware explicitly deletes the other with a `Set-Cookie` header that uses an expired date.
- **`userId` is never stored in a cookie:** It's derived from the JWT. The refresh-cookie name is only a write-time decision for where to store the refresh token.
- **Namespacing:** Cookies are suffixed with `siteId` (for example, `cc-nx_RefArch`) to support multi-site deployments, except `dwsid` and `dw_dnt`, which are excluded so external or B2C Commerce systems can read them directly.

User Type Detection

`userId` comes from the access-token JWT: registered tokens carry an `rcid` identity claim that guest tokens lack. It isn't determined by which refresh cookie exists. The refresh-cookie name is consulted only as a cold-start fallback, before any access token has been issued, and to decide which refresh cookie to write or delete on the response.



repeated JWT decoding in the hot path.

- **Refresh token:** Configurable through environment variables and capped at the B2C Commerce maximums (guest 30 days, and registered user 90 days).

401 Recovery Redirect

When a SCAPI call returns a `401` for a non-SLAS endpoint, the SCAPI client throws `AuthTokenInvalidError`. The middleware catches it in `handleAuthTokenInvalidation`, clears stale token state, re-runs the refresh or guest flow, and—if recovery succeeds—issues a `307` redirect back to the same URL so the request restarts with fresh cookies. The redirect carries the `x-sfnext-auth-recovery: 1` header for observability.

To prevent loops, a short-lived guard cookie, `cc-auth-recover (Max-Age=30)`, is set during recovery. If a `401` recurs while the guard is present, recovery isn't retried—the error surfaces and the response carries the `x-sfnext-auth-recovery-guard: 1` header. The guard is cleared on the follow-up request.

JWT Integrity Validation

SLAS guarantees `gcid` or `rcid` in the `isb` claim and `usid` in the `sub` claim. The middleware validates this on the incoming cookie token, but only when that token survived validation unchanged. After a refresh or guest login, re-validation is skipped because the freshly issued token was already validated inside `updateAuthStorageDataByTokenResponse` when it was stored.

If a structurally invalid token is detected (decodable but missing required claims, or undecodable), `AuthTokenInvalidError` is thrown and routed through the same recovery flow as a `401`: clear cookies, fresh login, `307` redirect. Because this indicates a critical token-issuance failure, it logs at the error level for production visibility.

Configuration

Environment Variables (Optional)

```
1 # Override guest refresh token expiry (cappe
2 PUBLIC_COMMERCE_API_GUEST_REFRESH_TOKEN_EXPI
3
4 # Override registered refresh token expiry (
5 PUBLIC_COMMERCE_API_REGISTERED_REFRESH_TOKEN
```



A leading-dot value scopes the cookie to the apex domain and all subdomains:

```
1 // config.server.ts - app.commerce.sites
2 sites: [
3   {
4     id: "RefArch",
5     cookies: { domain: ".example.com" }, //
6     defaultLocale: "en-US",
7     // ...
8   },
9 ];
```

Cookie attribute defaults (`path: '/'`, `sameSite: 'lax'`, `secure: true`) are applied automatically. The site `cookies.domain` takes precedence over them.

Accessing Session Data

On the Server

The auth middleware reads cookies from the request and builds the full `SessionData`, including tokens. You can access this data only on the server—in loaders, actions, and other middlewares. Use `getAuth(context)` :

```
1 import { getAuth } from "@/middlewares/auth."
2 import type { LoaderFunctionArgs } from "rea
3
4 export async function loader({ context }: Lo
5   const auth = getAuth(context);
6
7   const accessToken = auth.accessToken; // s
8   const customerId = auth.customerId;
9   const userType = auth.userType; // 'guest'
10  const usid = auth.usid;
11
12  return { customerId, isRegistered: userTyp
13 }
```

In Client Components

There's no client-side `getAuth`. Client components read the non-sensitive slice through the `useAuth()` hook—tokens are never available on the client:

```
1 import { useAuth } from "@/providers/auth";
2
3 function AccountBadge() {
4   const auth = useAuth(); // PublicSessionDa
```



```
10 }
```

For route-level auth checks, branch inside the server `loader`—not in a client guard.

Updating and Destroying Auth

Updating Auth (Login)

`updateAuth(context, updater)` accepts either a SLAS token response or a function updater:

```
</> | |
1 import { updateAuth, loginRegisteredUser } from '@middlewares/auth'
2 import type { ActionFunctionArgs } from '@remix-run/server-runtime'
3
4 export async function action({ request, context }: ActionFunctionArgs) {
5   const formData = await request.formData()
6   const email = formData.get('email') as string
7   const password = formData.get('password') as string
8
9   try {
10    const tokenResponse = await loginRegisteredUser({ email, password })
11    updateAuth(context, tokenResponse) // tokenResponse is a SLAS token response
12    return redirect('/account')
13  } catch {
14    return { error: 'Login failed' }
15  }
16 }
```

The token-response form re-derives `userType`, `customerId`, `usid`, and expiry from the new JWT, preserves the tracking-consent cookie, and writes the appropriate cookies. The function form, `updateAuth(context, (data) => ({ ...data, codeVerifier }))`, merges in fields such as the PKCE code verifier without a full token swap.

Destroying Auth (Logout)

```
</> | |
1 import { destroyAuth } from '@middlewares/auth'
2
3 export async function action({ context }: ActionFunctionArgs) {
4   destroyAuth(context) // clears storage; no cookies are set
5   return redirect('/')
6 }
```

Custom Auth Operations

The following server-side SLAS helpers are exported from `auth.server.ts`:



```
5 authorizePasswordless,  
6 getPasswordLessAccessToken,  
7 getPasswordResetToken,  
8 resetPasswordWithToken,  
9 requestOtp,  
10 verifyOtp,  
11 flashAuth,  
12 clearInvalidSessionAndRestoreGuest,  
13 } from "@middlewares/auth.server";
```

For the related shopper features, see [Social Login](#), [Passwordless Login](#), and [Password Reset](#).

Authentication Flows

New Guest User

1. The user visits with no auth cookies.
2. The server middleware finds no usable token and calls SLAS guest login.
3. The server writes `cc-nx-g`, `cc-at`, and `usid` (and any `dwsid` from the response).
4. The root loader serializes `clientAuth`, and `AuthProvider` makes it available through `useAuth()`.

Returning User (Token Valid)

1. The server reads cookies from the `Cookie` header.
2. If the access-token `exp` is in the future, the tokens are used as-is.
3. If the access token is expired but a refresh token exists, the server refreshes and rewrites `cc-at` (and `usid` if it changed).

Guest to Registered (Login)

1. The action calls `loginRegisteredUser()`, and SLAS returns a registered token whose JWT carries `rcid`.
2. `updateAuth(context, tokenResponse)` derives `userType = 'registered'` from the JWT.
3. The middleware writes `cc-nx` and deletes `cc-nx-g` to maintain mutual exclusivity.
4. Guest resources are merged into the registered account (see below).

Logout

1. The action calls `destroyAuth(context)`.



fresh guest login.

Guest to Registered Resource Merge

On any guest-to-registered transition (social, passwordless, or standard login), guest-owned resources must be captured *before* the token swap, because `customerId` changes from the guest `gcid` to the registered `rcid`, and SCAPI rejects the guest customer ID under a registered token:

```
1 // lib/api/auth/social-login.server.ts (shap
2 const guestWishlistSnapshot = await captureC
3 const result = await loginIDPUser(context, /
4 await mergeBasket(context); // AFTER swap
5 await mergeWishlist(context, guestWishlistSr
```

Tracking consent is preserved across the swap: `updateAuthStorageData` restores the `dw_dnt` cookie value (the source of truth) after clearing storage, and login and refresh calls forward it to SLAS as the `dnt` parameter.

Hybrid Storefronts (B2C Commerce Session Bridge)

There's no client-side cookie sync. The bridge is the `dwsid` cookie:

1. The SDK extracts `dwsid` from the SLAS response's `Set-Cookie` header, and the middleware persists it.
2. `dwsid` (and `dw_dnt`) aren't namespaced, so the B2C Commerce cartridge can read and write them directly.
3. The browser sends these cookies on every request. On the next full request, the React storefront's server middleware reads them from the `Cookie` header. There's no real-time iframe or single-page-app sync in middleware scope.

For more about hybrid implementations, see [Use Storefront Next in a Hybrid Implementation](#).

Hydration

Auth is available immediately during server-side rendering (SSR) and hydration without serializing any tokens:

- **Server:** The root loader builds the full `SessionData` through `getAuth(context)`, then returns only `clientAuth =`



`value={clientAuth}>` directly. Components that read `useAuth()` see exactly the same data that produced the SSR markup, so there's no hydration gap and no bootstrap snapshot.

- **Subsequent navigations:** React Router re-invokes the root loader, which re-derives `clientAuth`. `AuthProvider` stays mounted, and the root memoizes the provider tree on `clientAuth`, so only its `value` changes and `useAuth()` consumers re-render.

```
1 // providers/auth.tsx
2 export const AuthContext = createContext<PublicSessionData>()
3 export const useAuth = (): PublicSessionData => {
4   // ...
5 }
6 // root.tsx (shape)
7 const session = getAuth(context); // full SessionData
8 const clientAuth = getPublicSessionData(session)
9 // ...
10 <AuthProvider value={clientAuth}>{/* app */}
```

The server-side auth during hydration keeps a single source of truth, exposes no tokens to the client, and maintains a stable provider tree.

Type Safety

`SessionData` and `PublicSessionData` are custom types defined in the template, not part of React Router's [sessions and cookies API](#). Storefront Next doesn't use React Router's

`createCookieSessionStorage` because auth requires individual cookies with separate expiry times—access tokens expire in minutes, while refresh tokens persist for days. React Router's session API packs all data into a single cookie, which doesn't support these requirements.

```
1 import type { SessionData, PublicSessionData } from './types'
2
3 // Full server-side session (tokens included)
4 type SessionData = {
5   accessToken?: string;
6   accessTokenExpiry?: number;
7   refreshToken?: string;
8   refreshTokenExpiry?: number;
9   customerId?: string;
10  userType?: "guest" | "registered";
11  usid?: string;
12  encUserId?: string;
13  codeVerifier?: string; // OAuth2 PKCE (server-side)
14  idpAccessToken?: string;
15  idpAccessTokenExpiry?: number;
16  idToken?: string; // OIDC id_token (server-side)
17  idpRefreshToken?: string; // social login
18  dwsid?: string; // hybrid storefront session
```



```
24 sessionData,  
25   "customerId" | "userType" | "usid" | "encl  
26 >;
```

Best Practices

- **Server versus client:** Use `getAuth(context)` in loaders and actions, and `useAuth()` in components. Tokens are server-only.
- **No direct cookie writes:** Always go through `updateAuth()` or `destroyAuth()` –the middleware owns cookie serialization.
- **User-type checks:** Use `auth.userType`, which is JWT-derived.
- **Token refresh:** Handled automatically by the middleware. Don't refresh manually in routes.
- **Capture before swap:** On guest-to-registered login, snapshot guest resources before the token swap and merge after.
- **Security:** Never log or expose `accessToken` or `refreshToken`.
- **No `clientLoader` or `clientAction`:** Route modules use the server `loader` and `action` only.

For a comparison with the PWA Kit approach, see [Storage and Sessions Differences with PWA Kit](#).

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)



Try for free



MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

Component Library
APIs
Trailhead
Sample Apps
AppExchange

Events and Calendar
Partner Community
Blog
Salesforce Admins
Salesforce Architects

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)  [Your Privacy Choices](#)

